

Lab Sheet: Distributed Software Development with GitHub

LO1. - Describe the software engineering of programs using concurrency and synchronization and the development of distributed software

Objective:

This lab will introduce you to the fundamentals of **distributed software development** using **GitHub**. You will learn how to use Git for version control, collaborate on projects in a distributed team, and manage source code effectively across different locations.

You will also understand the basics of distributed software development principles, such as **branching**, **merging**, and **collaborating** using a **remote repository** on **GitHub**.

Tools Required:

- **Git** (installed on your local machine)
 - **GitHub** (Sign up for a free account at [GitHub](https://github.com))
 - **GitHub Desktop** (optional, for GUI-based Git management)
 - **Text Editor** (e.g., **VSCode**, **Atom**, or **Notepad++**)
-

Pre-Requisites:

- Basic knowledge of **Git** commands (clone, commit, push, pull).
 - Understanding of **version control** and the importance of managing code in a distributed team environment.
 - Basic understanding of **branches** and **merging** in Git.
-

Lab Tasks:

Part 1: Setting Up Git and GitHub Repository

1. **Install Git:**
 - Download and install Git from [Git's official website](https://git-scm.com).
 - After installation, open a terminal and verify by typing:
`git --version`
2. **Create a GitHub Repository:**
 - Log in to your **GitHub** account.
 - Create a new repository by clicking on **New** (top-right) and providing a name for your repository (e.g., `distributed-software-lab`).

- Optionally, initialize it with a **README.md** file to explain the purpose of your repository.
-

Part 2: Cloning the Repository and Making Changes

1. Clone the Repository:

- Clone the repository from GitHub to your local machine using **Git**. In the terminal, run the following command:
- `git clone https://github.com/your-username/distributed-software-lab.git`
- Navigate to the repository folder:
- `cd distributed-software-lab`

2. Create a New Branch:

- In a distributed software project, each feature or bug fix is developed in a separate branch.
- Create a new branch to work on a specific task. For example, if you are adding a new feature:
- `git checkout -b feature/new-feature`

3. Add and Commit Changes:

- Make some changes in your code (e.g., create a simple Java program or modify a `README.md` file).
- Stage the changes to be committed:
- `git add .`
- Commit the changes with a descriptive message:
- `git commit -m "Added a new feature"`

4. Push Changes to GitHub:

- Push your changes to the remote repository on GitHub:
 - `git push origin feature/new-feature`
-

Part 3: Collaborating with Other Developers

1. Collaborating by Creating Pull Requests:

- On GitHub, go to your repository and switch to the branch you pushed (e.g., `feature/new-feature`).
- You will see an option to **Create Pull Request**. Click it to initiate a pull request (PR) that requests to merge your changes into the `main` branch.

2. Review and Merge Pull Requests:

- If you are working with a team, collaborators will review your pull request, and after approval, they will merge it into the `main` branch.
 - To merge, click on **Merge Pull Request** and confirm.
-

Part 4: Handling Merge Conflicts

1. Simulate a Merge Conflict:

- Create two branches (`feature/branch1` and `feature/branch2`) from the `main` branch. Modify the same line of code in both branches (e.g., change the `README.md` file).
- Push both branches to GitHub.

2. Merge the Branches and Handle Conflict:

- Attempt to merge the branches using a pull request or directly using Git. Git will flag the conflict and require manual resolution.
 - In the terminal, run:
 - `git checkout main`
 - `git pull origin main`
 - `git merge feature/branch1`
 - `git merge feature/branch2`
 - Git will show conflict markers in the file. Open the file and decide how to resolve the conflict.
 - After resolving the conflict, add and commit the changes:
 - `git add .`
 - `git commit -m "Resolved merge conflict"`
 - `git push origin main`
-

Part 5: Syncing with Remote Repository**1. Pulling the Latest Changes:**

- In distributed development, it is important to frequently sync your local repository with the remote one to get updates made by others.
- To pull the latest changes from the remote repository:
 - `git pull origin main`

2. Push Your Final Changes:

- After completing your task and resolving conflicts, push your final changes back to GitHub:
 - `git push origin main`
-

Lab Submission:**1. GitHub Repository Link:**

- Share the link to your GitHub repository for review.

2. Documenting Workflow:

- Include a **README.md** file that explains how you completed each task in the lab (e.g., how you created branches, made commits, handled conflicts, etc.).
- List any challenges you faced and how you resolved them (e.g., how you resolved merge conflicts).

3. Push All Changes:

- Ensure that all the changes you made during the lab (code, branch management, commit history) are pushed to GitHub.
-

Additional Topics to Explore:

1. **Advanced Branching Strategies:**
 - Learn about advanced strategies like **Git Flow** and **GitHub Flow** for handling feature development, hotfixes, and releases.
 2. **Continuous Integration (CI):**
 - Explore how **GitHub Actions** or other CI tools like **Travis CI** can automatically run tests and deploy applications when changes are pushed.
 3. **Forking Repositories:**
 - Explore the process of forking repositories and contributing to open-source projects on GitHub.
-

Conclusion:

This lab introduces you to the fundamental concepts of **distributed software development** using **GitHub**. By completing this lab, you will be able to manage and collaborate on software development projects in a distributed environment, using version control and GitHub's collaboration tools effectively. You will also gain hands-on experience in resolving conflicts, creating branches, making commits, and using pull requests in GitHub.